

Présentation

OptiPick — Allocation intelligente de commandes en entrepôt

Notre objectif est de résoudre un problème d'optimisation logistique : comment affecter des commandes à différents agents en respectant des contraintes métier, tout en optimisant un critère global.

Problème

Comment assigner des commandes à des agents en respectant :

Capacité poids / volume

Zones interdites

Produits fragiles

Poids max par item

Incompatibilités produits

Objectifs

- Maximiser le nombre de commandes assignées
- OU
- Minimiser le coût total

- C'est un problème d'allocation sous contraintes, typique des CSP (Constraint Satisfaction Problems)

Architecture du système

Étapes

- -
 -
 - Chargement JSON
 - Parsing en objets Python
 - Enrichissement (poids total, volume, localisation)
 - Allocation (glouton ou MiniZinc)
 - Affichage des résultats
-
- Le projet est structuré proprement : séparation données, modèle, contraintes et solveur.
Cela permet de changer d'algorithme sans modifier toute l'architecture.

Architecture du système

```
{  
  "id": "R1",  
  "type": "robot",  
  "capacity_weight": 20,  
  "capacity_volume": 30,  
  ...  
},
```

```
raw = {  
    "id": "R1",  
    "type": "robot",  
    "capacity_weight": 20,  
    "capacity_volume": 30  
}
```

Le fichier `src/loader.py` lit le JSON et crée des objets Python `Agent` :

```
```python  
def build_agent(raw: dict) -> Agent:
 """Construit un agent à partir des données JSON."""
 return Agent(
 id=raw["id"],
 type=raw.get("type", "unknown"),
 capacity_weight=float(raw.get("capacity_weight", 0.0)), # ← Ici
 capacity_volume=float(raw.get("capacity_volume", 0.0)),
```

# Architecture du système

```
Résultat : Chaque objet `Agent` a un attribut `capacity_weight` (ex.
`agent.capacity_weight = 20`)
```

## ### 3. Passage à MiniZinc : `minizinc\_solver.py`

Le code Python qui appelle MiniZinc extrait les valeurs depuis les objets `Agent` et les passe au modèle :

```
```python  
from minimizinc import Instance, Model, Solver  
  
def allocate_with_minizinc(orders, agents, products, warehouse, solver_name):  
    # Charger le modèle  
    model = Model("models/allocation.mzn")  
    instance = Instance(solver, model)  
  
    # Extraire capacity_weight de chaque agent  
    capacity_weight = [agent.capacity_weight for agent in agents]  
    # Exemple : [20.0, 35.0, 20.0, 35.0, 50.0, 50.0]  
  
    # Passer la valeur à MiniZinc  
    instance["capacity_weight"] = capacity_weight  
  
    # Résoudre  
    result = instance.solve()
```


Modélisation des données

Modèle conceptuel

•
•
•
•

Agent
Order
Product
Warehouse

- $\text{assignment}[\text{order}_i] = \text{agent}_j$: Chaque commande choisit un agent.
- On modélise le problème sous forme déclarative : on ne dit pas comment affecter, on décrit les contraintes que doit respecter la solution.

Modélisation des données

```
% PARAMÈTRES
int: n_orders;          % Nombre de commandes
int: n_agents;          % Nombre d'agents
set of int: ORDERS = 1..n_orders;
set of int: AGENTS = 1..n_agents;

% Capacités des agents
array[AGENTS] of float: capacity_weight;
array[AGENTS] of float: capacity_volume;
array[AGENTS] of int: agent_type; % 0=robot, 1=human, 2=cart

% Poids et volumes des commandes
array[ORDERS] of float: order_weight;
array[ORDERS] of float: order_volume;

% Restrictions des robots
% Zones interdites: matrice booléenne [agent, zone] = true si zone interdite
% Zones: 0=A, 1=B, 2=C, 3=D, 4=E
int: n_zones = 5;
set of int: ZONES = 0..n_zones-1; %Les zones sont numérotées à partir de 0
array[AGENTS, ZONES] of bool: forbidden_zones; % true si zone interdite pour ce
array[AGENTS] of bool: no_fragile; % Pas d'objets fragiles
```

Algorithme glouton : First-Fit

Principe

Résultat

- On parcourt les commandes
- On assigne au premier agent disponible
- 30/30 commandes assignées
- Distance proxy = 594

- Avantages : Rapide - Simple
- Limites : Ne vérifie pas toutes les contraintes - Pas optimal

Algorithme glouton : First-Fit

JOUR 1 – Allocation naïve (First-Fit)

Commandes totales : 30

Commandes assignées: 30

Commandes non assignées: 0

Distance totale estimée (proxy): 594

Détail par agent:

– R1 (robot)

commandes: 7 -> ['Order_001', 'Order_002', 'Order_003', 'Order_004', 'Order_005', 'Order_006', 'Order_022']

poids: 18.64/20.00 kg (93.2%)

volume: 29.74/30.00 dm³ (99.1%)

vitesse: 2.0 m/s

– R2 (robot)

commandes: 5 -> ['Order 007', 'Order 008', 'Order 009', 'Order 010', 'Order 023']

MiniZinc & CSP

Constraint Satisfaction Problem

CSP

Variable - Domaines - Contraintes

MiniZinc

- Modélisation déclarative
- Solveur explore l'espace des solutions

Résultat

- 9/10 commandes assignées
- Toutes contraintes respectées
- Meilleure validité métier

- Contrairement au glouton, MiniZinc explore les combinaisons possibles et garantit que toutes les contraintes sont respectées.

Modélisation des contraintes

MiniZinc

```
(venv) romain@MacBook-Air-de-Romain optipick % python main.py --test --minizinc --solver chc  
✂ Utilisation de MiniZinc pour l'allocation optimale...
```

Add to Chat 🗨️

JOUR 2 – Allocation optimale avec MiniZinc

Commandes totales : 5
Commandes assignées: 2
Commandes non assignées: 3
Distance totale estimée (proxy): 12

Détail par agent:

- R1 (robot)
 - commandes: 2 -> ['Order_002', 'Order_003']
 - poids: 0.35/20.00 kg (1.7%)
 - volume: 1.80/30.00 dm³ (6.0%)
 - vitesse: 2.0 m/s

Commandes non assignées:

- Order_001
- Order_004
- Order_005

Explication des contraintes

```
% 1. Capacité en poids
constraint forall(agent_idx in AGENTS) (
    sum(order_idx in ORDERS where assignment[order_idx] == agent_idx) (order_weight[order_idx]) <= capacity_weight[agent_idx]
);
%On parcourt chaque agent un par un.
%Pour chacun, on additionne le poids de toutes les commandes qui lui sont assignées.
%On impose que ce poids total ne dépasse pas la capacité maximale de l'agent.
```

```
% 2. Capacité en volume
constraint forall(agent_idx in AGENTS) (
    sum(order_idx in ORDERS where assignment[order_idx] == agent_idx) (order_volume[order_idx]) <= capacity_volume[agent_idx]
);
```

```
% 3. Restrictions des robots (zones interdites)
% Note: order_zones[order_idx] = 0 peut signifier Zone A ou zone non définie (par défaut Zone A)
% On vérifie seulement si la zone est dans la plage valide [0..4]
constraint forall(order_idx in ORDERS, agent_idx in AGENTS) (
    (assignment[order_idx] == agent_idx /\
    agent_type[agent_idx] == 0 /\
    order_zones[order_idx] >= 0 /\
    order_zones[order_idx] < n_zones) ->
    (not forbidden_zones[agent_idx, order_zones[order_idx]])
);
```

Explication des contraintes

```
array[ORDERS] of var 0..n_agents: assignment;
```

```
% 4. Restrictions des robots (pas d'objets fragiles)
constraint forall(order_idx in ORDERS, agent_idx in AGENTS) (
    (assignment[order_idx] == agent_idx /\ agent_type[agent_idx] == 0 /\ no_fragile[agent_idx]) ->
    (not order_has_fragile[order_idx])
);
```

```
% 5. Restrictions des robots (poids max par item)
constraint forall(order_idx in ORDERS, agent_idx in AGENTS) (
    (assignment[order_idx] == agent_idx /\ agent_type[agent_idx] == 0 /\ max_item_weight[agent_idx] > 0) ->
    (order_max_item_weight[order_idx] <= max_item_weight[agent_idx])
);
```


Explication des contraintes

```
% 6. Incompatibilités entre produits
% Si deux commandes sont incompatibles, elles ne peuvent pas être assignées au même agent
constraint forall(order_first in ORDERS, order_second in ORDERS where order_first < order_second /\
    incompatible[order_first, order_second]) (
    assignment[order_first] != assignment[order_second] \/
    assignment[order_first] == 0 \/
    assignment[order_second] == 0
);
```

```
% 7. EXTENSION 1 : Restrictions multi-niveaux (picking)
% Les robots (agent_type=0) ne peuvent accéder qu'aux niveaux 1-2.
% Une commande contenant des produits sur niveaux 3, 4 ou 5 doit être assignée à un humain.
constraint forall(order_idx in ORDERS, agent_idx in AGENTS) (
    (assignment[order_idx] == agent_idx /\ agent_type[agent_idx] == 0) ->
    (not order_has_high_level[order_idx])
);
```

Explication des contraintes

```
array[AGENTS] of bool: agent_available;  
array[ORDERS] of bool: order_available;
```

```
% 9. EXTENSION 3 : Agents indisponibles (pannes, pauses)  
% Un agent indisponible ne peut recevoir aucune commande  
% Si agent_available[agent_idx] = false, alors aucune commande ne peut lui être assignée  
constraint forall(order_idx in ORDERS, agent_idx in AGENTS) (  
    (assignment[order_idx] == agent_idx) ->  
    agent_available[agent_idx]  
);
```

```
% 10. EXTENSION 3 : Commandes en rupture de stock  
% Une commande en rupture de stock ne peut pas être assignée  
% Si order_available[order_idx] = false, alors la commande doit rester non assignée (assignment = 0)  
constraint forall(order_idx in ORDERS) (  
    (not order_available[order_idx]) ->  
    (assignment[order_idx] == 0)  
);
```

Valeurs de sortie

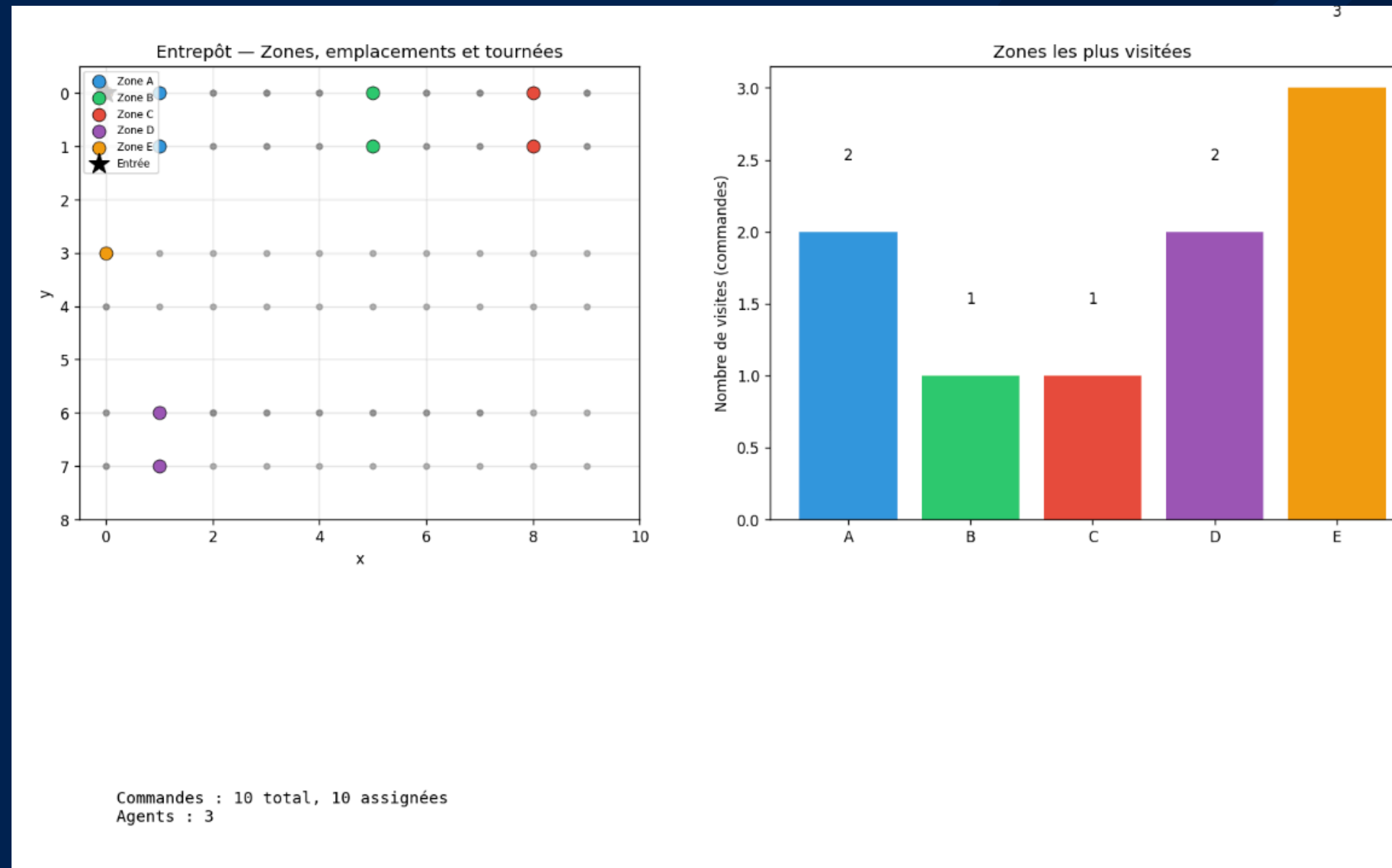
Si tu lances MiniZinc en ligne de commande (ex: `minizinc allocation.mzn instance.dzn`):

```
output [  
    "assignment = ", show(assignment), ";\n",  
    "num_assigned = ", show(num_assigned), ";\n",  
    "num_express_assigned = ", show(num_express_assigned), ";\n",  
    "weighted_objective = ", show(weighted_objective), ";\n",
```

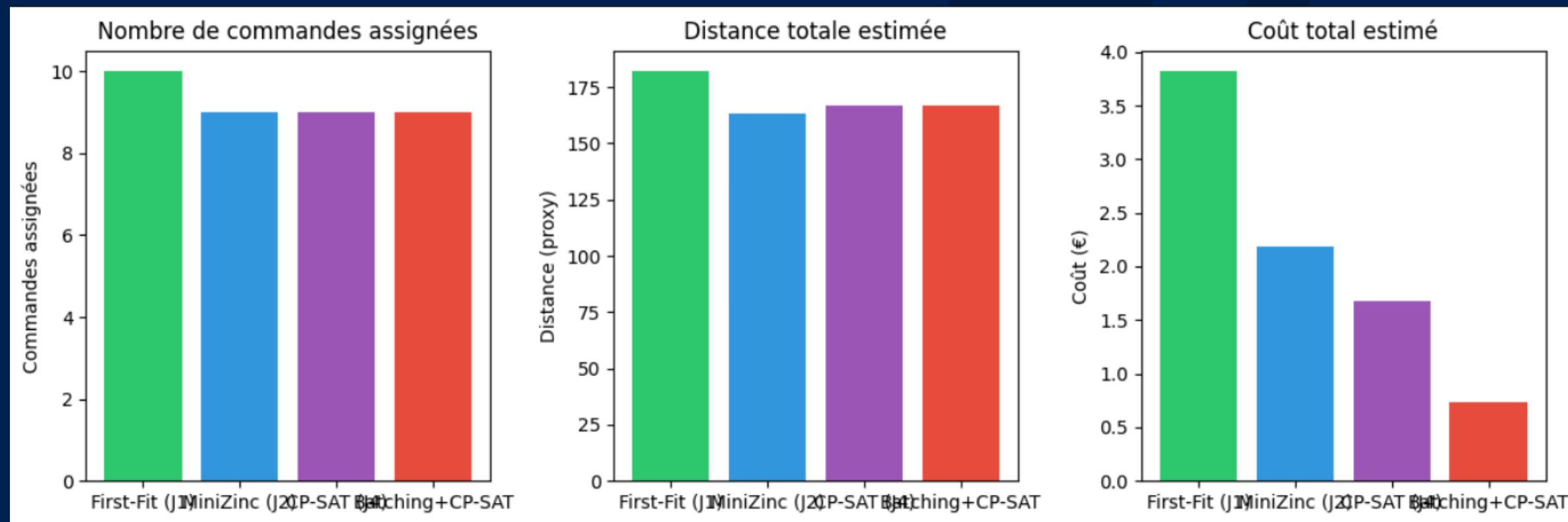
```
python main.py --minizinc --solver gecode
```

```
assign_arr = result["assignment"]  
if hasattr(assign_arr, "__iter__") and not isinstance(assign_arr, (str, dict)):  
    assign_list = list(assign_arr)  
else:  
    assign_list = [assign_arr]  
  
assignment = {}  
for i, order in enumerate(orders):  
    idx = i if i < len(assign_list) else 0  
    val = assign_list[idx] if idx < len(assign_list) else 0  
    if val and 1 <= val <= n_agents:  
        assignment[order.id] = agents[val - 1].id  
    else:  
        assignment[order.id] = None  
  
return assignment
```


Valeurs de sortie du test



Valeurs de sortie du test



Architecture

```

└─ optipick
  ├── __pycache__
  ├── .mpl
  ├── .streamlit
  └─ data
    ├── agents.json
    ├── orders.json
    ├── products.json
    ├── test_agents_3.json
    ├── test_agents_3diff.json
    ├── test_agents.json
    ├── test_orders_10.json
    ├── test_orders.json
    └── warehouse.json
  ├── docs
  ├── examples
  │   ├── README_test.md
  │   ├── test_ortools.py
  │   ├── test.dzn
  │   └── test.mzn
  ├── image_présentation
  ├── models
  │   └── allocation.mzn
  ├── results
  ├── scripts
  ├── src
  ├── static
  └── templates
```

```

└─ scripts
  ├── plot_day4_metrics.py
  └─ src
    ├── __pycache__
    ├── __init__.py
    ├── allocation_cpsat.py
    ├── allocation.py
    ├── batching.py
    ├── constraints.py
    ├── day4_comparison.py
    ├── day5_dashboard.py
    ├── day5_human_robot.py
    ├── day5_patterns.py
    ├── day5_simulation.py
    ├── day5_storage.py
    ├── loader.py
    ├── minizinc_solver.py
    ├── models.py
    ├── optimization.py
    ├── routing.py
    ├── storage.py
    ├── utils.py
    └── visualization.py
  ├── static
  │   ├── css
  │   └── js
  ├── templates
  └── index.html
```

```

└─ templates
  └─ venv
    ├── bin
    ├── etc
    ├── include
    ├── lib
    ├── share
    ├── pyvenv.cfg
    ├── .gitignore
    ├── app_streamlit.py
    ├── app.py
    ├── DEPLOY_STREAMLIT.md
    ├── ENONCE_PROJET_OPTIPICK.t...
    ├── INSTALLATION.md
    ├── main.py
    ├── README.md
    ├── requirements.txt
    └── setup_venv.sh
```

Merci à tous !